

Trip Planner — Week 8 Summary

Jiaxuan Wu

2026-05-19

Trip Planner — Week 8 Summary

Repo: <https://github.com/superjwu/trip-planner> **Vercel:** <https://trip-planner-theta-wheat.vercel.app/>

TL;DR

Most of this week was the “*Phase B is actually visible and bearable*” sprint. Phase B (multi-stop combos) shipped to master last week, but two things hadn’t happened yet: the **DB migration wasn’t applied**, so the new code crashed with Could not find the 'stop_snapshots' column; and multi-stop made fresh generations **15–45s** instead of the old 6–9s, with no UI acknowledgment of the new wait. Both fixed. Also caught a Next 16 routing regression that was hiding the sign-in page, redesigned the progress bar so it doesn’t look frozen during long compute, and added hover prefetch on pick cards so clicking into a destination usually feels instant.

What landed (by commit)

08e7367 — GeneratingProgress: flowing gradient + log-asymptote pacing

The old bar eased out to 92% in 10 seconds and then **froze for the remaining 20–35s** of real LLM work. New curve is $96 \times (1 - e^{(-t/18)})$ so it keeps moving smoothly to ~96% over ~60s and never visually stalls. The fill is now a coral↔slate gradient with background-size: 200% animating L→R every 2.5s, plus a pulsing coral dot glowing at the leading edge. Elapsed-seconds counter beside the kicker.

df65baf — Wait-time copy on three surfaces + drop sign-in workaround

The earlier silence around compute time was the biggest source of “feels broken” complaints. Added wait copy in three places:

- **/plan wizard** — italic line below the submit button: *“Usually 20–40 seconds — we ask ChatGPT to rank routes against your preferences and score the tradeoffs.”* Swaps to a *don't refresh* variant while pending. zh parity included.
- **GeneratingProgress** — range hint: *“Most trips finish in 20–40s. Multi-stop combos can run to ~50s.”* When the user has been waiting >50s it swaps to a reassurance line so the asymptote plateau feels intentional.
- **Trips list** — pending row reads *“Ready to compute — opens in ~20–40s”*; computing row reads *“Picking destinations... ~20–40s”*.

Also dropped a stale Turbopack workaround (`sign-in/page.tsx + sign-up/page.tsx`) that Next 16.2.6 was rejecting as a same-specificity conflict with the optional catch-all. The bare `/sign-in` URL now resolves to the Clerk widget again.

fd2ab42 — Itinerary: skeleton day-rows + hover prefetch on pick cards

Two layers of improvement for the focused-pick view, where the day-by-day LLM call is **another** ~6–12s wait:

- **ItineraryDraftingSkeleton** — instead of one *“Drafting your itinerary...”* line, render **N numbered day cards** matching the trip length (clamped 1–14, default 5). Each row has a coral placeholder circle + three text-line skeletons with 90ms staggered animation delays so a wave runs down the page. Italic header: *“Drafting your day-by-day · usually 6–12 seconds.”*
- **HoverPrefetchItinerary** — new client wrapper around each pick card. On mouseenter or focus, fires `ensureItinerary` in the background. The action is idempotent at the DB layer, so it costs nothing for already-drafted itineraries. Net effect: by the time the user clicks, the LLM call has been running for ~300–800ms — often enough time to finish for a short trip. Click-to-render feels near-instant on hover-able devices; the existing `ItineraryAutoFetch` on the focused page is still the fallback for touch / keyboard / fast clickers.

Migration 20260513230221 multi_stop_combos (applied this week)

Phase B added `recommendations.stops` (jsonb) and `recommendations.stop_snapshots` (jsonb) plus a backfill that turns every pre-existing rec into a 1-stop combo. The code was on master since last week but the migration hadn't been applied to the live Supabase project — so opening any trip threw `Could not find the 'stop_snapshots' column`. Applied via the Supabase MCP after granting the `mcp_supabase_apply_migration` permission in `.claude/settings.local.json`. Verified backfill: every existing recommendation row now has a valid 1-stop array.

Why generation got slower — and how we addressed it

Phase B's multi-stop is genuinely more expensive at the LLM layer:

1. Bigger prompt — +~1500 tokens for the new stop rules + nearby (≤ 350 mi) lists per candidate.
2. More reasoning per pick — the model now evaluates stop *combinations* (1–3 stops $\times$ {slug, order, days}), not single destinations.
3. Bigger structured output — `stops[]` array adds ~30% output tokens.

4. Cold cache — REC_PROMPT_VERSION bumped to rec-v8-multi-stop invalidated every prior cached rec.

Typical timings on the new path: **15–25s for single-stop, 25–45s for multi-stop, <500ms cached**. Per-stop hydration (drive estimator runs in parallel) is negligible.

We didn’t make the LLM faster — we made the wait *legible*. Between the new pacing curve, the explicit time hints, the day-row skeleton, and the hover prefetch, the experience went from “*is this stuck?*” to “*OK, it’s working; here’s roughly how long.*”

Stats

- **8** Supabase migrations applied (latest 20260513230221 multi_stop_combos).
- **0** uncaught route errors after the Next 16.2.6 sign-in routing fix.
- **329** destinations enriched, npm run audit:meta clean.
- **2** LLM flows on gpt-5.5 (rec engine medium, itinerary low).
- **0** lint errors, 9 pre-existing warnings (gallery only, intentional).
- **0** committed secrets — verified via the security review last week, unchanged.

Commits this week

Hash	Subject
fd2ab42	itinerary: skeleton day-rows + hover prefetch on pick cards
df65baf	ui: wait-time copy on 3 surfaces + drop sign-in workaround
08e7367	GeneratingProgress: flowing gradient + log-asymptote pacing
677ba7d	fix(auth): Next 16 sign-in/sign-up bare-segment 404

(Plus the multi_stop_combos migration applied to the live Supabase project, no code commit.)

Still in the backlog

Audit-derived punch list from this week’s planning. Pick order roughly by user-felt impact:

1. **Finish Phase B’s multi-stop UI surfaces** — DestinationCard.tsx and ExpandedDestination.tsx still render only the anchor stop visually, even though the stops[] array is now in the data. The plan called for companion-stop chips (“+ Savannah · 2 days”) and a “The Route” section. Without these, Phase B’s whole point is invisible. ~4–6 hr.
2. **Specific error messages + retry button** — failed compute renders a generic “Something went wrong”. friendlyComputeError likely collapses “ChatGPT not connected” / rate-limited / invalid JSON into the same string, and there’s no retry button on the failed state. ~1.5 hr.

3. **Per-stop hydration** — weather + cost are still fetched only for the anchor. Will become more obvious once #1 ships. ~3 hr.
4. **Cache-hit visibility** — `rec_cache` hits are silent. A small “*served instantly · cached*” pill next to “Why these four” would make the warm-cache speedup intelligible. ~1 hr.
5. **Rebase / merge feature/shiny-vip-badge** — branched from before Phase B; needs to rebase before merging. ~10 min.

How to run locally

```
npm install
cp .env.local.example .env.local      # fill in Clerk, Supabase,
                                       Codex keys
npm run dev                            # http://localhost:3000
```

Pre-commit hook (Phase B hygiene) automatically runs:

```
npx tsc --noEmit
npm run lint
npm run audit:meta
```

Generated 2026-05-19. See CLAUDE.md for the project guide and docs/sprints/v2-v3-plan.md for sprint context.